

## FIELD NOTES

# The agent handoff problem

Four AI coding agents shared one rule set and one work queue. Only one of them could actually drive the workflow — so when it ran out of quota mid-task, everything stopped. Here is what it took to fix that, and the eight things that turned out to be wrong along the way.

---

4

agents sharing one rule layer

4

slices to make handoff safe

8

assumptions that did not survive review

## THE SETUP

### Shared text was the easy half

The starting point was ordinary: a canonical repo holding rule files and skill definitions, symlinked into each agent's config directory. One source of truth, four consumers — a CLI-based coding agent, a second CLI agent from a different vendor, an IDE-based agent, and an editor plugin.

That part worked. What did not work was *knowing when to apply any of it*. Two of the four agents had an empty entry file — the document their runtime reads at session start to learn which rules exist. They had the whole rule library on disk and no index into it.

The practical result: every task routed back through the one agent that **did** load the rules, because it was the only one that knew they were there. That agent became the orchestrator by accident, not by design.

Fixing that is a rendering problem — generate each agent's entry file from one routing table. An afternoon's work. The interesting problem was what it exposed underneath.

THE REAL PROBLEM

# State lived in a conversation, not a file

With the routing fixed, any agent could read the rules. None of them could *pick up work in progress*. Which branch, which worktree, which step, who was holding it, what had already been verified — all of it existed only in the orchestrating agent's context window.

The obvious fix is a file-based work queue, and one already existed: a task manifest per job, with commands to claim, hand off, and close. That was the state of things when the work started, and the backlog note said the remaining job was to build exactly that queue.

**The backlog note was wrong.** The queue existed. What it did not have was any notion of *safe* takeover — and that gap is invisible until two agents touch the same task.

WHAT WAS ACTUALLY BROKEN

# Four holes in a queue that looked finished

| COMMAND     | WHAT IT DID                                              | WHY THAT FAILS                                                                            |
|-------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| claim       | Refused only when the task was in the in-progress state. | An agent dying during verify left a task owned but claimable. Two agents, one task.       |
| handoff     | Cleared the owner field unconditionally.                 | No caller check — anyone could release anyone's task. Takeover with no lock.              |
| set --state | Accepted any state, including the terminal one.          | Walked straight past the quality gate that the dedicated close command enforced.          |
| gate        | Read one global verification file.                       | Not bound to task, branch, commit or diff — a stale result could close an unrelated task. |

Underneath all four: the verification runner itself returned **pass** when every check was skipped, because it counted failures and found none. A repository with no test script configured produced a

green verdict.

## THE FIX

# Borrow the primitives from distributed systems

None of this is novel. Treat each agent as an unreliable worker competing for a shared record, and the standard answers apply directly.

---

### SLICE Leases and compare-and-swap

1 Ownership expires. Every mutation carries the version it expects; a lockfile serialises writers and the version is re-read inside the lock. Takeover requires naming the current owner *and* an expired lease. A typed manifest schema makes the required fields explicit instead of conventional.

---

### SLICE Migration, and splitting the verifier

2 The schema change needed a migration path — see the lessons below for why that was urgent. Separately, the verification runner was split: it now runs read-only and prints structured output, and a *different* party writes the result into the task record.

---

### SLICE A controller that emits the next action

3 One read-only command reads the manifest, resolves the role through the existing role-assignment tool, and emits a single machine-readable action: who acts, with which model and sandbox, which command to run, and what marks it complete. An agent joining mid-task asks the controller instead of reading the whole procedure.

---

### SLICE An adapter that actually spawns the agent

4 For the one agent invocable from a shell, the adapter really runs it — passing through the model, effort and sandbox named in the action, wrapping the prompt in a filesystem boundary, and applying the state change only on a pass verdict. For the agents that cannot be scripted, it writes a ready-to-paste prompt file and says so.

---

# Three parties touch a verification, never one

The original design had a single command run the checks and write the verdict. That quietly violated the project's own policy that a verifier holds no write tools — and it meant the agent that wrote the code could also record the judgement on it.

```
verifier    → runs checks read-only, prints JSON to stdout
controller → ingests that JSON, writes the signed artifact
owner       → reads the artifact, records the gate verdict

constraint: controller ≠ executor and controller ≠ verifier
```

The artifact is bound to task id, repository, branch, commit and a hash of the diff, and expires after an hour. Feed it to a different task, a different branch, or a tree that has moved since, and the gate refuses it.

A pleasant side effect: the verifier can now run in a fully read-only sandbox, because it no longer needs to write anything at all.

## CLOSING CONDITION

# A drill, not a checklist

The reviewing agent set the bar for calling this done, and it was a good bar: interrupt the orchestrator at *every* transition, have a different agent take over without hand-editing state, and reach completion.

That drill lives in the repository and runs on demand. Six simulated interruption points, plus one scenario behind a flag that spends real credits and calls a real agent end to end:

```
[TAKEOVER] agent-a → agent-b    state=in-progress    PASS
AGENT_RESULT {"verdict":"pass", ...}
CAS_TRANSITION=APPLIED expectedVersion=4
DRILL PASS  before=in-progress@4 after=verify@5 next=review
```

The live scenario is off by default. A drill that silently bills you every time it runs is a drill people stop running.

# Eight assumptions that did not survive

These are the parts worth stealing. Every one of them produced a green result at the time, and every one was wrong.

## DEAD CONFIGURATION

### A patch to a field nothing reads

A restrictive instruction was traced to a configuration field, the field was edited, and the docs said to re-render. It changed nothing — **no code consumed that field**. The live string was hardcoded elsewhere entirely.

Grep for the *consumer*, not just the definition. A setting with no reader is a comment.

## ORDER OF OPERATIONS

### The guardrail rewrote itself into a trap

Templates resolve placeholders *before* per-agent path rewriting. A boundary naming a directory got its path rewritten per agent — so the rendered instruction told one agent **not to read its own working directory**.

The fix was not better wording. It was emitting a sentinel and substituting the real text after rewriting, which deletes the whole bug class.

## TEST DESIGN

### An allowlist exempts whatever you forget

The test that checked "every skill invoking the agent carries the boundary" held a hand-written list of skills. Four call sites had shipped with no boundary at all, **silently excused by not being on the list**.

Replacing it with a scan for actual invocations found a fifth that two separate reviews had missed.

## CONTROL GROUP

### Absent from the failure list is not passing

Comparing failures before and after a change, one test appeared only in the "after" list — apparently a regression. Running it alone against the clean tree showed **it failed there too**. It had never run in the baseline, because a hook timed out and took its whole block with it.

A missing row means unknown, not green.

#### SCOPE OF EVIDENCE

### One test file is not the suite

A change was validated by running the one obviously-related test file and declaring the remaining failures pre-existing environment noise. The full comparison against a clean checkout found **two real regressions in files that had never been run** — plus a size budget the change had quietly blown.

Building the clean baseline was itself instructive: without the compiled artifacts linked in, it reported 642 failures and was worthless as a control.

#### MIGRATION

### Count the live records first

Tightening the record schema used a reasonable-sounding policy: read old records loosely, reject writes to them. In the abstract, fine. In practice there were **eight live task records, one mid-flight** — and no migration command existed.

The queue froze. Everything in it became readable and untouchable at the same time.

#### VERIFICATION BOUNDARY

### An agent cannot verify its own escape hatch

The adapter that spawns a nested agent could not be tested by that agent: its own sandbox sets a variable blocking outbound network, so the nested call always failed. It correctly reported blocked rather than downgrading to a simulation.

**The proof had to come from outside the sandbox.** Worth knowing before you design a self-test.

#### DEFINITION OF DONE

### Artifact exists ≠ failover works

The original assessment checked that role chains were declared, that a state file existed, that commands had help text — and marked the item solved. The reviewing agent named it precisely: **substituting "the artifact is present" for "the failure path works."**

Nothing had ever been interrupted and resumed. That is why the closing condition became a drill.

# What stays manual, and why that is written down

Two of the four agents cannot be driven by a script. Their task-spawning and user-prompting capabilities are runtime features of the host application, not command-line entry points. No amount of adapter work changes that.

So the adapter does not pretend. For those hosts it writes a prompt file and prints a banner saying the step is manual by design — with the reason, and an explicit line stating that this is not a temporary limitation and that nothing was spawned.

A stub that *looks* like automation is worse than an honest manual step, because the next person to read the logs cannot tell the difference. Name the seam.

Also still manual, on purpose: committing and merging. The role table lists a human there, and the controller stops at that step rather than handing it to anyone.

## IF YOU BUILD ONE

# The short version

**Shared rules are a rendering problem.** One routing table, generated per agent, checked for drift. The hard part is never the text.

**Handoff is a concurrency problem.** Leases, compare-and-swap, an explicit transition graph, and verification results bound to the exact tree they describe. Borrow from distributed systems; the failure modes are identical.

**Separate the parties.** Whoever wrote the code does not record the verdict on it. Whoever runs the checks does not need write access to do so.

**Close reliability work with a drill.** Interrupt it at every transition and make something else finish the job. A checklist of declared capabilities proves nothing.

**Let the tooling stop and ask.** When no eligible reviewer exists — because the only candidate wrote the code and the alternate is offline — the correct output is a halt with a reason, not a silent skip.

Written up from a working multi-agent harness: four agents, one shared rule layer, a file-based work queue with leases and compare-and-swap, and a crash-resume drill that has to pass before any of it is called finished. Organisation-specific names, paths and identifiers have been removed.